

Atividade — Desafio de Sobrevivência

1. Objetivo

Desenvolver, utilizando React, um pequeno jogo de sobrevivência baseado em turnos, utilizando os Hooks estudados durante as aulas, como `useState`, `useEffect` e `useCallback`.

O jogador deverá administrar seus recursos, tomar decisões durante a partida e tentar alcançar uma das condições de vitória antes que sua vida ou energia chegue a zero.

2. Estado inicial do jogador

Ao iniciar uma nova partida, o jogador deverá possuir:

- **Vida:** 100
 - **Energia:** 100
 - **Comida:** 5
 - **Recursos:** 0
-

3. Ações do jogador

A cada rodada, o jogador poderá realizar uma ação.

As ações disponíveis serão:

- Explorar
- Descansar
- Comer
- Trabalhar

Porém, haverá uma regra de sequência para controlar a utilização das ações.

4. Regra de sequência das ações

O jogador poderá utilizar no máximo **duas ações diferentes entre Explorar**.

Depois que o jogador realizar duas ações, a próxima ação obrigatoriamente deverá ser **Explorar**.

Exemplo:

1ª ação → Comer
2ª ação → Trabalhar
3ª ação → Explorar

Depois da exploração, as ações **Comer, Trabalhar e Descansar** ficam novamente disponíveis.

Outro exemplo:

1ª ação → Descansar
2ª ação → Comer
3ª ação → Explorar

Após Explorar:

Descansar → disponível
Comer → disponível
Trabalhar → disponível
Explorar → disponível

O sistema deverá impedir que o jogador clique em uma terceira ação antes de realizar a exploração.

Durante o bloqueio, os botões de **Descansar, Comer e Trabalhar** deverão permanecer desabilitados, enquanto o botão **Explorar** deverá ficar disponível.

5. Explorar — Evento aleatório

A ação **Explorar** não deverá possuir um resultado fixo.

Ao clicar em **Explorar**, o sistema deverá sortear aleatoriamente um dos eventos abaixo.

Encontrar comida

Comida: +2

Mensagem:

Você encontrou comida abandonada!

Encontrar recursos

Recursos: +10

Mensagem:

Você encontrou madeira e outros materiais úteis.

Perder vida

Vida: -45

Mensagem:

Você sofreu um acidente durante a exploração.

Perder energia

Energia: -40

Mensagem:

Você ficou exausto durante a exploração.

Nada acontece

Nenhum atributo deverá ser alterado.

Mensagem:

Você explorou a região, mas não encontrou nada.

6. Descansar

Ao utilizar a ação **Descansar**, os atributos deverão ser alterados da seguinte maneira:

Energia: +30

Vida: +5

Mensagem sugerida:

Você descansou e recuperou suas forças.

A energia e a vida não deverão ultrapassar o valor máximo de **100**.

7. Comer

Ao utilizar a ação **Comer**, os atributos deverão ser alterados:

Comida: -1

Vida: +20

Mensagem sugerida:

Você comeu e recuperou parte da sua vida.

O jogador somente poderá utilizar a ação caso possua pelo menos **1 unidade de comida**.

A vida não deverá ultrapassar **100**.

8. Trabalhar

Ao utilizar a ação **Trabalhar**, os atributos deverão ser alterados:

Energia: -25

Recursos: +10

Mensagem sugerida:

Você trabalhou e conseguiu obter novos recursos.

O jogador deverá possuir energia suficiente para realizar a ação.

9. Histórico da partida

A aplicação deverá possuir um histórico mostrando todas as ações e eventos ocorridos durante a partida.

Exemplo:

Histórico

Você comeu e recuperou parte da sua vida.

Você trabalhou e conseguiu obter novos recursos.

Você encontrou madeira e outros materiais úteis.

Você descansou e recuperou suas forças.

Você encontrou comida abandonada!

O histórico deverá ser atualizado a cada ação realizada.

10. Condição de vitória

O jogador poderá vencer a partida ao atingir uma das seguintes condições:

Vitória por recursos

Recursos $\geq$ 50

Mensagem:

Você venceu! Conseguiu reunir 50 recursos.

Assim que uma dessas condições for atingida, o jogo deverá ser encerrado e as ações deverão ser desabilitadas.

Deverá ser exibida uma mensagem informando que o jogador venceu.

11. Condição de derrota

A partida deverá terminar quando a **Vida** ou a **Energia** chegar a zero ou ficar abaixo de zero.

Condição:

Vida $\leq$ 0
OU
Energia $\leq$ 0

Mensagem:

Game Over! Você não conseguiu sobreviver.

Após o Game Over, todas as ações deverão ser desabilitadas.

Deverá ser exibido um botão:

Iniciar Nova Partida

12. Nova partida

O botão **Iniciar Nova Partida** deverá restaurar todos os atributos para os valores iniciais:

Vida: 100

Energia: 100
Comida: 5
Recursos: 0

Também deverá:

- Limpar o histórico;
- Remover o estado de vitória;
- Remover o estado de Game Over;
- Liberar novamente os botões;
- Reiniciar a sequência de ações;
- Salvar o novo estado no `localStorage`.

13. Salvamento com `localStorage`

O jogo deverá utilizar `localStorage` para salvar o estado atual da partida.

Deverão ser armazenados, no mínimo:

- Vida;
- Energia;
- Comida;
- Recursos;
- Histórico;
- Estado da partida;
- Controle da sequência de ações.

Ao atualizar a página, o sistema deverá recuperar os dados salvos e permitir que o jogador continue a partida de onde parou.

Caso não exista uma partida salva, o jogo deverá iniciar com os valores padrão.

14. Uso dos Hooks do React

A aplicação deverá utilizar os Hooks estudados como `useState`, `useEffect` e `useCallback`.

15. Regras gerais do jogo

O jogo deverá seguir as seguintes regras:

1. O jogador começa com 100 de vida.
2. O jogador começa com 100 de energia.
3. O jogador começa com 5 unidades de comida.
4. O jogador começa com 0 recursos.
5. O jogador poderá realizar apenas duas ações entre cada exploração.
6. Após duas ações, a próxima ação obrigatoriamente deverá ser Explorar.
7. Explorar deverá gerar um evento aleatório.
8. Os eventos de Explorar poderão beneficiar ou prejudicar o jogador.
9. O jogador não poderá comer se não possuir comida.
10. O jogador não poderá realizar ações que exijam mais energia do que possui.
11. A vida máxima será 100.
12. A energia máxima será 100.
13. O jogador vence ao atingir 50 recursos.
14. O jogador perde quando a vida ou energia chegar a zero ou ficar abaixo de zero.
15. Após vitória ou derrota, as ações deverão ser desabilitadas.
16. O jogador deverá poder iniciar uma nova partida.
17. O estado da partida deverá ser salvo no `localStorage`.
18. Ao atualizar a página, a partida deverá continuar de onde parou.
19. Todas as ações deverão ser registradas no histórico.
20. A interface deverá informar claramente ao jogador os eventos ocorridos e o estado atual da partida.

16. Resultado esperado

Ao final da implementação, o jogo deverá apresentar uma experiência de sobrevivência baseada em decisões, eventos aleatórios e gerenciamento de recursos.

O jogador deverá precisar escolher estrategicamente suas ações, controlar sua vida, energia, comida e recursos, respeitar a regra de exploração a cada três turnos e tentar alcançar uma das condições de vitória antes de sofrer Game Over.

Entrega

A atividade será realizada individualmente e será corrigida individualmente.

O objetivo é demonstrar que você consegue utilizar os Hooks estudados para controlar o comportamento e o estado de uma aplicação React.